TOOLKIT · BACKEND ENGINEERING HANDBOOK

PROMETHEUS COMPLETE HANDBOOK

Time-Series Metrics, PromQL, Alerting Rules, Alertmanager, SLIs/SLOs & Production Reliability

Phase	III — Production Observability
Tool ID	10 of 13
Curriculum Scope	Complete 2-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/prometheus/ (01 and 02 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, SREs, Systems Engineers
Engineering Focus	Metrics Scrapes, Core Metric Types, PromQL Math, Alert Rules, Alertmanager Routing, Recording Rules & SLO Incident Triage

This comprehensive handbook provides an authoritative, production-grade guide to **Prometheus** for backend and site reliability engineering. Covering three rigorous pedagogical layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips engineers to design resilient metrics collection pipelines, understand time-series data structures and scrape intervals, leverage the four core metric types (Counters, Gauges, Histograms, Summaries), master PromQL expressions (rate, increase, sum by, histogram_quantile), implement Spring Boot Actuator and Micrometer instrumentation, write actionable alerting rules, configure Alertmanager deduplication and routing, precompute expensive aggregations with recording rules, define quantitative SLIs and SLOs with burn rates, prevent high-cardinality label explosion, and systematically triage real-world production incidents.

Curriculum Structure & Core Topics

Part I — Metrics, Time Series & Monitoring Fundamentals: Time-series data architecture, metric names and label dimensions, the four core metric types (Counter, Gauge, Histogram, Summary), pull-based scraping mechanics, scrape targets, jobs, instances, exporters vs native instrumentation, Metrics vs Logs vs Traces, Spring Boot Actuator setup, PromQL basics (up, rate, increase, error ratios, histogram quantiles), and cardinality management.

Part II — Alerting, SLOs, Recording Rules, Production Reliability & Incident Mastery: Complete alerting pipeline, anatomy of alerting rules (expr, for, labels, annotations), Alertmanager architecture (grouping, deduplication, routing, inhibition, silencing), SLIs vs SLOs vs SLAs, error budgets and burn rates, symptom vs cause alerting, recording rules, cardinality explosion prevention, security, and production incident runbooks.

PART I: METRICS, TIME SERIES & MONITORING FUNDAMENTALS

Chunk 1 of 2 · Backend Engineering Handbook

Prometheus — Chunk 1 of 2: Metrics, Time Series & Monitoring Fundamentals

Backend Engineering Tools & Infrastructure Handbook

Learning Objectives

By the end of this chapter, you should be able to:

- Explain what Prometheus does and why backend engineers use it.
- Understand metrics, labels, time series, samples, and scrape intervals.
- Distinguish counters, gauges, histograms, and summaries.
- Configure Prometheus to scrape a Spring Boot application's metrics.
- Write basic PromQL queries for request rates, errors, latency, and resource usage.
- Understand exporters, service discovery, and instrumentation.
- Debug missing targets, missing metrics, and misleading dashboards.
- Explain how metrics differ from logs and distributed traces.

Chunk 1 establishes the foundations and hands-on workflow. Chunk 2 covers production alerting, SLOs, recording rules, scaling, reliability, security, incident response, and advanced interview mastery.

Layer 1 — Fundamentals & Mental Models

1. What Is Prometheus?

Prometheus is an open-source monitoring and alerting toolkit designed around numerical time-series data.

Backend engineers use it to answer questions such as:

- How many HTTP requests is my API receiving?
- What percentage of requests are failing?
- Is API latency increasing?
- Is the application running out of memory?
- Are database connections becoming saturated?
- Is a background worker falling behind?
- Did a deployment cause a performance regression?

Prometheus collects metric samples, stores them as time series, and provides a query language called PromQL.

A typical architecture looks like this:

```
Spring Boot API ■■■■■■
Node Exporter ■■■■■■■■■■> Prometheus ■■> PromQL queries
Redis Exporter ■■■■■■■■         |
Other services ■■■■■■■■         +■■> Alerting rules
                                      |
                                      v
                                Alertmanager
                                      |
                                      v
                               Notification channels
```

Grafana can query Prometheus and visualize its data. Alertmanager handles alert grouping, routing, and notification workflows when configured.

Prometheus is not a replacement for application logs, distributed tracing, or a full incident-management process. Each provides different evidence.

2. The Mental Model: A Time Series

A metric is a numerical measurement. A time series is a sequence of samples identified by a metric name and its label set.

Consider:

```
http_requests_total{
  method="GET",
  route="/orders",
  status="200"
} 1250
```

This says the current recorded counter value for that particular combination of labels is `1250`.

Prometheus stores timestamped samples over time, conceptually:

Timestamp	Value
10:00:00	1200
10:00:15	1215
10:00:30	1230
10:00:45	1250

A query can analyze this history rather than only the latest value.

The four building blocks

- **Metric name:** what is measured, such as `http_requests_total`.
- **Labels:** dimensions describing the series, such as method or status.
- **Sample:** a numerical value with a timestamp.
- **Scrape interval:** how frequently Prometheus retrieves metrics from a target.

Why labels matter

The following are different time series:

```
http_requests_total{method="GET",status="200"}
http_requests_total{method="POST",status="201"}
http_requests_total{method="GET",status="500"}
```

Each distinct combination of metric name and labels identifies a separate series.

Labels enable filtering and aggregation, but too many distinct combinations create high cardinality. That can increase memory consumption, storage requirements, and query cost.

Rule: Use labels for bounded dimensions, not arbitrary user-specific values.

Good label candidates:

- HTTP method.
- Normalized route.
- Status class.
- Service name.
- Environment.

Dangerous label candidates:

- User ID.
- Email address.
- Request ID.
- Raw URL containing arbitrary IDs.
- Unbounded search queries.

A route label should generally use a normalized route such as `/orders/{id}`, rather than `/orders/839472`.

3. The Four Core Metric Types

Prometheus instrumentation commonly uses counters, gauges, histograms, and summaries.

Counter

A cumulative value that increases over time and can reset when the process restarts.

- **Characteristics:** Monotonically increasing (can reset to 0 on application restart). Never decreases during normal operation.
- **Typical examples:** Total requests served, processed background jobs, failed payment attempts, bytes transmitted.
- **Typical PromQL query:**

```
rate(http_requests_total[5m])
```

Gauge

A value that can arbitrarily increase or decrease over time.

- **Characteristics:** Snapshot of current state. Goes up or down at any time.
- **Typical examples:** Active concurrent requests, queue depth, resident memory usage, active database connections, CPU temperature.
- **Typical PromQL query:**

```
process_resident_memory_bytes
```

Histogram

Observations are sampled into configurable, cumulative buckets. Histograms support latency distribution analysis and percentile calculations.

- **Characteristics:** Exposes `<basename>_bucket{le="..."}`, `<basename>_count`, and `<basename>_sum`. Allows aggregation across multiple target instances.
- **Typical examples:** HTTP request duration, database query execution duration, payload sizes.
- **Typical PromQL query:**

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(http_server_request_duration_seconds_bucket[5m])
  )
)
```

Summary

Records observations and exposes direct client-calculated quantiles (such as ϕ=0.5, 0.9, 0.99) along with observation count and sum.

- **Characteristics:** Quantiles are computed on the client side. Unlike histograms, client-computed summary quantiles cannot be aggregated across multiple instances.
- **Typical examples:** High-accuracy percentile tracking on a single instance where bucket boundaries are unknown in advance.

Counter vs gauge: the common interview trap

A counter is cumulative. A gauge is a current measurement.

Do not use a counter to represent the number of active requests because that value can decrease. Use a gauge for the current number of in-flight requests.

Likewise, don't interpret a counter's absolute value as the current request rate. Use `rate()` or `increase()` over an appropriate time range.

4. Pull-Based Monitoring: How Scraping Works

Prometheus commonly uses a pull model: it periodically requests metrics from configured targets.

```
Prometheus                      Spring Boot
    |                               |
    |------ GET /actuator/prometheus->|
    |<----- metrics exposition -------|
    |                               |
    |------ wait for interval --------|
    |                               |
    |------ GET /actuator/prometheus->|
```

The endpoint returns a metrics exposition format that Prometheus can parse.

A scrape configuration might look like:

```
scrape_configs:
  - job_name: "orders-api"
    metrics_path: "/actuator/prometheus"
    static_configs:
      - targets: ["localhost:8080"]
```

Here, `localhost:8080` means the address from Prometheus's network perspective. If Prometheus runs in a separate container, `localhost` usually refers to the Prometheus container itself, not the Spring Boot container.

Why pull-based collection is useful

- Prometheus controls collection intervals.
- Targets can expose metrics without implementing a push pipeline.
- Target health is observable through scrape status.
- The same model works with applications and exporters.

Does Prometheus always require pull?

No. Prometheus-compatible systems can integrate with push-oriented workflows through components such as the Pushgateway for certain short-lived jobs. It is not a general replacement for scraping long-running services, and it should not be used as an arbitrary event-ingestion system.

5. Scrape Targets, Jobs, and Instances

A **target** is an endpoint Prometheus scrapes.

A **job** groups targets of a similar role.

An **instance** normally identifies an individual target, often through its address.

Example:

```
scrape_configs:
  - job_name: "orders-api"
    static_configs:
      - targets:
          - "orders-api-1:8080"
          - "orders-api-2:8080"
```

Prometheus can then distinguish the two instances while allowing queries to aggregate both.

This is useful for identifying whether:

- One instance is unhealthy.
- One replica has much higher latency.
- A particular deployment is generating errors.
- Only a subset of targets is being scraped.

6. Metrics, Logs, and Traces Are Different

The three pillars of observability provide complementary signals during an investigation:

Telemetry Pillar	Concrete Example	Primary Diagnostic Role
Metrics (*Prometheus*)	*"Error rate rose to 8% at 10:05."*	**Aggregation & Trends:** Best for real-time rates, system health indicators, capacity tracking, and automated threshold alerts.
Logs	*"Order 481 failed because the database connection timed out."*	**Context & Root Cause:** Best for discrete event records, stack traces, diagnostic error messages, and payload inspection.
Traces	*"The request spent 900 ms waiting for PostgreSQL after a 20 ms proxy hop."*	**Path & Bottleneck Analysis:** Best for distributed request propagation, inter-service call graphs, and pinpointing slow hops across microservices.

These systems complement one another.

For example, Prometheus alerts you to a latency spike. Logs help explain the affected requests, and traces can reveal which dependency consumed most of the time.

7. What Is an Exporter?

An exporter exposes metrics about a system through an endpoint Prometheus can scrape.

Examples include:

- Node Exporter for host-level metrics.
- Redis Exporter for Redis-related metrics.
- PostgreSQL exporters for database metrics.
- Blackbox Exporter for probing endpoints and network services.

An exporter translates system-specific information into a format Prometheus can collect.

Application instrumentation vs exporters

Application instrumentation measures application behavior directly, such as request duration or business-operation counters.

Exporters commonly expose metrics from systems that do not natively expose the required Prometheus metrics.

You may need both. Host CPU metrics cannot, by themselves, explain why a specific API endpoint is slow.

Layer 2 — Practical Engineering & Workflows

8. Run Prometheus Locally with Docker

Create a project directory:

```
mkdir prometheus-lab
cd prometheus-lab
```

Create a file named `prometheus.yml`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

Create `compose.yaml`:

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus-lab
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"

volumes:
  prometheus_data:
```

For a reproducible or production deployment, pin a tested image version or digest rather than relying indefinitely on `latest`.

Start it:

```
docker compose up -d
docker compose logs -f prometheus
```

Open the Prometheus web interface at:

`http://localhost:9090`

Check target health through **Status** → **Targets**.

You should see the Prometheus target become healthy if the container and configuration are correct.

Useful operational commands

```
docker compose ps
docker compose logs --tail=100 prometheus
docker compose restart prometheus
```

To stop the lab:

```
docker compose down
```

This preserves the named data volume. To remove it too, you would need to explicitly remove the volume, which deletes stored time-series data.

9. Configure Spring Boot Metrics

For a Spring Boot application, Actuator and Micrometer are common ways to expose application metrics.

Add the appropriate dependencies to your Maven project:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Use dependency versions managed by your Spring Boot project's dependency management unless you have a specific compatibility reason not to.

Configure the metrics endpoint in **`application.yml`**:

```
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus
  endpoint:
    health:
      show-details: never
```

This example exposes health and Prometheus endpoints through the configured management web endpoint. Review management-port, network, and authentication settings before deployment.

Start the application and test:

```
curl -i http://localhost:8080/actuator/health
curl -i http://localhost:8080/actuator/prometheus
```

The Prometheus endpoint should return a text exposition of the metrics available through the registry and instrumentation.

If the endpoint returns **`404`**, check whether the dependencies are present, the application started successfully, and the endpoint is exposed at the path you expect.

10. Connect Prometheus to Spring Boot

Update `prometheus.yml`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "orders-api"
    metrics_path: "/actuator/prometheus"
    static_configs:
      - targets: ["host.docker.internal:8080"]
```

`host.docker.internal` is commonly available in Docker Desktop environments to reach the host from a container. The appropriate address differs by operating system and deployment setup. In Docker Compose, using the backend's service name on a shared network is often cleaner.

Reload or restart Prometheus after updating the file, depending on its reload configuration:

```
docker compose restart prometheus
```

Then inspect **Status** → **Targets**. The `orders-api` target should show as `UP` when the endpoint is reachable and the scrape succeeds.

What if the target is `DOWN`?

Check in this order:

1. Does the Spring Boot application run?
2. Does the metrics endpoint respond directly?
3. Can the Prometheus container resolve and reach the target address?
4. Is the port correct?
5. Does the application expose the metrics endpoint at the configured path?
6. Does the Prometheus log explain a connection, parsing, or HTTP error?

Avoid changing multiple settings simultaneously. Verify each layer separately.

11. Your First PromQL Queries

PromQL is Prometheus's query language.

Open the Prometheus expression browser and try the following queries.

Query 1 — Is the target scrape healthy?

```
up
```

Prometheus usually exposes `up` for configured scrape targets:

- `1` means the last scrape succeeded.
- `0` means the last scrape failed.

Filter to one job:

```
up{job="orders-api"}
```

Query 2 — Inspect available metrics

```
{job="orders-api"}
```

This selector is useful for exploring series belonging to the job, although a metric-name selector is more precise once you know what to inspect.

For a specific metric, try:

```
process_resident_memory_bytes
```

Metric availability depends on the application's dependencies, instrumentation, and configuration.

Query 3 — Request rate

For an HTTP request counter:

```
rate(http_server_requests_seconds_count[5m])
```

This is a common metric pattern in Micrometer-based Spring applications. Exact names and labels depend on the Spring Boot and Micrometer versions and the instrumentation being used.

The query estimates the per-second rate of increase over a five-minute window.

Query 4 — Request volume over a window

```
increase(http_server_requests_seconds_count[1h])
```

This estimates the counter increase over the selected one-hour window. It can produce fractional estimates because Prometheus extrapolates counter behavior at the window boundaries.

Query 5 — Filter by status

Metric names and label names must match the actual exposed series. A common Micrometer HTTP metric includes a `status` label:

```
sum by (status) (
  rate(http_server_requests_seconds_count[5m])
)
```

This aggregates the observed request rate by status code.

If your instrumentation uses a different metric or label, inspect the available series and adapt the query.

Query 6 — Approximate 5xx error ratio

```
sum(
  rate(http_server_requests_seconds_count{status=~"5.."}[5m])
)
/
sum(
  rate(http_server_requests_seconds_count[5m])
)
```

This computes the share of requests represented by matching 5xx series, assuming the metric and label conventions match the query.

Handle the no-traffic case deliberately: the denominator may be zero or no series may be returned. Also ensure the numerator and denominator cover the same intended population of requests.

12. Understand Histograms and Percentiles

Latency averages can hide slow requests.

Imagine 100 requests:

- 95 complete in 50 ms.
- 5 complete in 2 seconds.

The average alone may not convey how bad the slowest requests feel to users.

Percentiles help describe a distribution:

- **p50:** median latency.
- **p95:** 95% of observations are at or below this value, according to the distribution estimate.
- **p99:** 99% are at or below this value.

If your application exposes classic Prometheus histogram buckets, a p95 query can look like:

```
histogram_quantile(
  0.95,
  sum by (le) (
    rate(http_server_requests_seconds_bucket[5m])
  )
)
```

`le` identifies the upper bound of each classic histogram bucket.

The query aggregates bucket rates across instances before estimating the percentile. This is important: calculating each instance's percentile and then averaging the percentiles is generally not equivalent to calculating a fleet-wide percentile.

Histogram caveats

- Bucket boundaries determine the quality of the estimate.
- A percentile is an estimate, not necessarily an exact observed duration.
- A metric may use a native histogram or a different instrumentation convention.
- The exact metric name depends on the library and version.
- Sparse traffic can make short-window percentiles unstable.

Engineering takeaway: Choose histogram boundaries and observation windows based on actual latency objectives and traffic volume.

13. Scrape Interval and Retention

The scrape interval determines how frequently Prometheus collects samples from targets.

```
global:
  scrape_interval: 15s
```

A shorter interval gives finer temporal resolution but increases collection overhead and the number of samples stored.

A longer interval reduces collection volume but can miss short-lived spikes and delays detection.

Retention controls how long historical data is kept. It depends on configuration, storage capacity, and operational requirements.

A simple storage intuition:

```
More targets
  × more time series per target
  × more samples per series
  × longer retention
  = more storage and processing demand
```

This is not a precise storage formula, because compression, churn, and other factors matter. It captures the variables you need to reason about.

14. Common Prometheus Problems

Problem A — Target is `DOWN`

Likely causes:

- Wrong host or port.
- Application is stopped.
- Metrics endpoint is not exposed.
- Container networking is incorrect.
- Authentication or TLS requirements are missing.
- The target returns an error or invalid exposition.

Start with `up`, the Targets page, and Prometheus logs.

Problem B — The metric doesn't exist

Possible causes:

- The relevant instrumentation is not installed.
- No request or operation has triggered the metric yet.
- The metric name differs from your assumption.

- The metric is not exposed by the registry.
- The metric changed across library versions.

Use the endpoint itself to inspect available metrics, then query the exact name.

Problem C — Dashboard values look wrong

Check:

- Units: seconds vs milliseconds, bytes vs megabytes.
- Counter vs gauge semantics.
- Whether the query uses `rate()` for a counter.
- Whether aggregation groups the correct labels.
- Whether the dashboard window matches the incident.
- Whether the metric includes every relevant instance.
- Whether missing series or no traffic is handled correctly.

A dashboard can be visually attractive and still be operationally misleading.

Layer 3 — Production Engineering & Interview Mastery

15. Production Scenario: Error Rate Rises After a Deployment

Scenario: Your Spring Boot API's 5xx error ratio rises immediately after a deployment.

Step 1 — Confirm the symptom

Query the error rate:

```
sum(
  rate(http_server_requests_seconds_count{status=~"5.."}[5m])
)
```

Compare it with total request rate:

```
sum(
  rate(http_server_requests_seconds_count[5m])
)
```

Step 2 — Compare the baseline

Inspect the same time window before and after deployment. Look at the affected routes, instances, and status codes where those dimensions are available.

Step 3 — Correlate with other signals

- Application logs: stack traces and exception messages.
- Traces: slow or failing downstream calls.

- Database metrics: connection-pool saturation and query latency.
- Infrastructure metrics: CPU, memory, and restarts.
- Deployment records: version and configuration changes.

Step 4 — Choose a mitigation

If evidence strongly connects the issue to the release, a controlled rollback may be safer than trying to repair production configuration through multiple unverified edits.

Step 5 — Verify recovery

Confirm that the error ratio returns toward baseline, latency stabilizes, and the intended version is running.

Lesson: Prometheus detects and quantifies a symptom. Root-cause analysis requires correlated evidence.

16. Production Scenario: Memory Is Increasing

Scenario: The process's resident memory rises over several hours.

Inspect the metric:

```
process_resident_memory_bytes
```

For a particular job:

```
process_resident_memory_bytes{job="orders-api"}
```

Then investigate:

1. Is memory rising across every instance or only one?
2. Did traffic volume increase?
3. Is the growth correlated with a particular workload?
4. Are containers approaching their memory limits?
5. Are garbage collection and heap metrics available?
6. Does the process stabilize or continue growing after traffic drops?
7. Do logs or heap diagnostics suggest retained objects or a leak?

Do not conclude that every rise is a memory leak. Caches, warm-up, workload, and normal memory-management behavior can all affect usage.

17. Production Scenario: Metrics Disappear

Scenario: A Grafana panel shows gaps even though the API appears healthy.

Investigate:

- Target scrape status.
- Prometheus restarts and logs.
- Application endpoint availability.
- Changes to metric names or labels after deployment.

- Prometheus configuration changes.
- Time-range and query-window selection.
- Resource pressure and scrape duration.
- Whether the metric series is created only after a relevant event.

A missing series is not always the same as a numerical zero. Dashboard and alert logic must account for missing data explicitly.

18. Cardinality and Cost Control

Cardinality is the number of distinct time series.

Consider:

```
http_requests_total{route="/orders/{id}",status="200"}
```

This uses a bounded route label.

By contrast:

```
http_requests_total{path="/orders/847239"}
```

If every request has a different path value, Prometheus may create a very large number of series.

High cardinality can lead to:

- Increased memory consumption.
- Higher storage usage.
- Slower queries.
- Increased ingestion overhead.
- Greater operational complexity.

Practical controls

1. Normalize route labels.
2. Avoid user IDs, raw URLs, request IDs, and arbitrary strings as labels.
3. Review newly introduced labels before deployment.
4. Track active series and cardinality growth.
5. Use logs or traces for high-cardinality event-level context.
6. Keep metric dimensions aligned with real operational questions.

Interview takeaway: Labels are dimensions, not a place to store arbitrary event data.

19. Monitoring Is Not the Same as Observability

Monitoring commonly asks whether a system is operating within expected bounds.

Observability uses available telemetry to help explain unfamiliar internal behavior.

Prometheus supports both, especially through metrics and derived indicators, but metrics alone may not identify the precise cause of a novel incident.

A mature backend setup combines:

- Metrics for trends, rates, and alerting.
- Logs for event-level details.
- Traces for request-path analysis.
- Deployment metadata for change correlation.
- Runbooks and operational procedures for response.

20. Interview Questions and Answers

Q1. What is Prometheus?

An open-source monitoring and alerting toolkit that collects, stores, and queries time-series metrics.

Q2. What is a time series?

A sequence of timestamped samples identified by a metric name and label set.

Q3. What is a scrape?

Prometheus's process of fetching metrics from a configured target endpoint.

Q4. What does `up` mean?

It is a target-health metric: `1` generally indicates the last scrape succeeded; `0` indicates failure.

Q5. Why use `rate()` on a counter?

It estimates the average per-second increase over a selected window and accounts for counter resets.

Q6. What is the difference between a counter and a gauge?

A counter accumulates increases and may reset. A gauge represents a value that can rise or fall.

Q7. What is a histogram?

A metric type that groups observations into buckets and records counts and sums, allowing distribution analysis and percentile estimation.

Q8. Why can averaging p95 values be misleading?

Percentiles are not generally additive or linearly composable. Aggregate histogram buckets first, then calculate the desired percentile for a fleet-wide estimate.

Q9. What is cardinality?

The number of distinct time series. High cardinality can increase memory, storage, and query costs.

Q10. Why avoid user IDs as metric labels?

They can create an unbounded number of time series. High-cardinality event details belong in other telemetry systems.

Q11. What is an exporter?

A component that exposes metrics from a system in a format Prometheus can scrape.

Q12. What is the difference between metrics and logs?

Metrics summarize numerical behavior over time. Logs capture individual events and their contextual details.

Q13. Why might a Prometheus target be down even though the service is healthy?

The scrape endpoint may be unreachable, incorrectly configured, inaccessible from the Prometheus network, or returning an error.

Q14. What does a shorter scrape interval change?

It improves temporal resolution and may reduce detection delay, but increases collection and storage overhead.

Q15. What is the role of Grafana?

Grafana queries data sources such as Prometheus and visualizes metrics through dashboards and related workflows.

Q16. Does Prometheus automatically know every metric in your application?

No. Metrics must be exposed through supported instrumentation or exporters and collected by configured targets.

Q17. What is the Pushgateway for?

It supports selected short-lived or batch jobs that cannot be scraped reliably during their lifetime. It is not a general-purpose replacement for scraping long-running services.

Q18. Why can a missing metric be different from zero?

A series may not exist because the event has never occurred, instrumentation is missing, or collection failed. Zero is an actual value; absence requires separate interpretation.

Q19. How would you investigate a latency spike?

Use PromQL to establish timing and scope, then correlate with application metrics, logs, traces, infrastructure, and deployment history.

Q20. Does a successful scrape prove the application is healthy?

No. It proves that the metrics endpoint was scraped successfully. The application can still return errors, be slow, or fail important business operations.

21. Hands-On Mastery Assessment

Complete this checklist to verify your foundational Prometheus competency:

- [ ] Run Prometheus locally and inspect its scrape target.
- [ ] Expose Spring Boot metrics through Actuator and Micrometer.
- [ ] Configure Prometheus to scrape the application.
- [ ] Explain the difference between a counter and a gauge.
- [ ] Use `up` to verify scrape health.
- [ ] Query a request counter with `rate()`.
- [ ] Calculate a 5xx ratio using the actual metric names and labels.
- [ ] Explain how histogram buckets support percentile estimation.
- [ ] Find and fix a deliberately incorrect target address.
- [ ] Explain why high-cardinality labels can destabilize monitoring.
- [ ] Distinguish metrics, logs, and traces in an incident.
- [ ] Document the first checks for a missing metric or target.

22. Chunk 1 Summary

You now have the foundations of Prometheus monitoring:

- Metrics and time series.
- Counters, gauges, histograms, and summaries.
- Pull-based scraping, targets, jobs, and exporters.
- Spring Boot Actuator and Micrometer integration.
- Core PromQL patterns for health, request rates, errors, and latency.
- Cardinality risks and practical debugging workflows.
- The relationship between metrics, logs, and traces.

Next: Prometheus — Chunk 2 of 2: Alerting, SLOs, Recording Rules, Production Reliability & Incident Mastery.

That chapter will complete the fixed two-chunk Prometheus curriculum, including alert rules, Alertmanager, actionable alert design, production scaling, security, and advanced incident scenarios.

PART II: ALERTING, SLOS, RECORDING RULES, RELIABILITY & INCIDENT MASTERY

Chunk 2 of 2 · Backend Engineering Handbook

Prometheus — Chunk 2 of 2: Alerting, SLOs, Recording Rules, Production Reliability & Incident Mastery

Backend Engineering Tools & Infrastructure Handbook

Source file:
`content/prometheus/02-prometheus-alerting-slos-recording-rules-production-reliability.md`

Learning Objectives

By the end of this chapter, you should be able to:

- Write actionable Prometheus alerting rules.
- Explain how Prometheus, Alertmanager, and notification channels work together.
- Design alerts around user impact rather than noisy infrastructure symptoms.
- Understand SLIs, SLOs, error budgets, and burn rates.
- Use recording rules to make expensive PromQL queries more efficient.
- Diagnose missing alerts, false positives, high-cardinality incidents, and monitoring failures.
- Explain monitoring reliability and alerting trade-offs in backend engineering interviews.

This chapter completes the three required layers of the Prometheus handbook:

1. **Fundamentals & Mental Models**
2. **Practical Engineering & Workflows**
3. **Production Engineering & Interview Mastery**

Layer 1 — Fundamentals & Mental Models

1. The Complete Monitoring Pipeline

Prometheus collecting metrics is only the beginning. Production monitoring must turn measurements into decisions and actions.

A typical monitoring pipeline looks like this:

```
Spring Boot Application
        |
        | Exposes /actuator/prometheus
        v
    Prometheus
        |
        | Evaluates PromQL alert expressions
        v
   Alerting Rules
        |
        | Sends firing alerts
        v
   Alertmanager
        |
        | Groups, deduplicates, routes
        v
 Email / Slack / PagerDuty / Webhook
```

Each component has a distinct responsibility.

Component	Responsibility
Application instrumentation	Produces application metrics
Exporter	Exposes metrics from a system that does not natively expose Prometheus metrics
Prometheus	Scrapes, stores, and queries time-series metrics
Alerting rules	Evaluates conditions and creates alerts
Alertmanager	Groups, deduplicates, silences, and routes alerts
Notification receiver	Delivers the notification to a person or system
Grafana	Visualizes metrics and dashboards; can also support alerting

Mental model: Prometheus detects a condition; Alertmanager manages the resulting notifications.

Prometheus does not inherently know whether a person has received or acted on an alert. Notification delivery and incident response are separate operational concerns.

2. What Is an Alerting Rule?

An alerting rule is a continuously evaluated expression that becomes active when its condition is true.

For example:

```
up == 0
```

This expression identifies targets whose most recent scrape indicates they are down.

A rule adds a name, evaluation behavior, labels, and descriptive annotations:

```
groups:
  - name: backend-availability
    rules:
      - alert: BackendTargetDown
        expr: up{job="backend-api"} == 0
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Backend API target is down"
          description: "Prometheus has been unable to scrape this target for the alert's
 pending period."
```

The important fields are:

- `alert`: the alert's name.
- `expr`: the PromQL condition.
- `for`: how long the condition must remain true before the alert fires.
- `labels`: metadata used for classification and routing.
- `annotations`: human-readable context.

Pending versus firing

Suppose the condition becomes true at 10:00 and the rule has `for: 2m`.

- At 10:00, the alert enters the pending state.
- If the condition remains true, it becomes firing after the pending period.
- If the condition clears before the period elapses, it returns to inactive.

This reduces alerts caused by brief fluctuations.

The `for` duration is not a universal solution to alert noise. If a serious failure requires immediate action, delaying it may be harmful.

3. What Is Alertmanager?

Prometheus evaluates rules. Alertmanager manages the alerts that are sent out.

Its core responsibilities include:

- **Grouping:** combine related alerts into one notification.
- **Deduplication:** avoid repeatedly notifying about equivalent alert states.
- **Routing:** direct alerts to appropriate receivers.
- **Inhibition:** suppress lower-priority notifications when a related higher-priority alert is firing.
- **Silencing:** temporarily suppress notifications matching specified conditions.

For example, if one database outage causes 40 API instances to fail, notifying an on-call engineer 40 separate times may create noise. Grouping can produce one notification with the relevant instance details.

Grouping should preserve useful context. Overly broad grouping can hide distinct incidents inside one notification.

4. Metrics Are Not the Same as Service Objectives

A metric tells you what happened. A service objective defines what level of service you intend to deliver.

Consider an API that receives 100,000 eligible requests in a measurement window.

If 99,500 requests succeed within the agreed latency threshold, the measured proportion is:

Formula: $SLI = \frac{99,500}{100,000} = 99.5\%$

Here is the terminology:

- **SLI — Service Level Indicator:** the measured indicator of service behavior, such as availability or latency.
- **SLO — Service Level Objective:** the target for that indicator, such as 99.9% successful requests over a rolling 30-day window.
- **SLA — Service Level Agreement:** an agreement with users or customers that may include commitments and consequences for missing them.
- **Error budget:** the amount of unreliability permitted by an SLO over its measurement window.

An SLO is not simply a dashboard threshold. It expresses an explicit reliability target.

Error budget example

Suppose an availability SLO is 99.9% over a 30-day window.

The allowed failure proportion is:

Formula: $1 - 0.999 = 0.001 = 0.1\%$

Using a 30-day window of 43,200 minutes, a time-based approximation of the budget is:

Formula: $43,200 \times 0.001 = 43.2 \text{ minutes}$

This time-based calculation is only an approximation when the SLI is request-based. If requests vary significantly over time, calculate the budget using eligible request counts instead.

The error budget creates a practical trade-off: when reliability is within budget, the team may have room for more change; when the budget is being consumed rapidly, the team may need to prioritize reliability work.

5. Alert on Symptoms and Causes

A useful monitoring system distinguishes user-visible symptoms from infrastructure causes.

Symptom-oriented alerts include:

- The API's successful-request ratio is falling.
- Request latency is exceeding the service objective.
- The error budget is being consumed too quickly.

Cause-oriented alerts include:

- CPU utilization is unusually high.
- A disk is approaching capacity.
- Kafka consumer lag is increasing.
- A database connection pool is nearly exhausted.

Both categories matter, but they serve different purposes. Symptom alerts help establish user impact; cause alerts help diagnose it.

A CPU alert by itself does not prove the service is unhealthy. High CPU can be expected during normal load. Conversely, an API may be returning errors while CPU remains low.

Production principle: Prioritize alerts that indicate a real problem requiring action. Use infrastructure alerts as supporting diagnostic signals.

Layer 2 — Practical Engineering & Workflows

6. Build a Reproducible Local Monitoring Lab

This lab assumes that you already have a Prometheus configuration and Spring Boot metrics endpoint from Chunk 1. Use the following as an extension, adapting paths and service names to your existing Compose project.

Recommended structure:

```
monitoring/
■■■ prometheus.yml
■■■ alert-rules.yml
■■■ alertmanager.yml
```

Step 1 — Configure rule evaluation

Add a rule-file reference and an evaluation interval to `prometheus.yml`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alert-rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]

  - job_name: backend-api
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["backend-api:8080"]
```

The target names must be resolvable from the Prometheus container's network. In Docker Compose, a service name usually works when the services share a network.

Step 2 — Create actionable alert rules

Create `alert-rules.yml`:

```
groups:
  - name: backend-availability
    rules:
      - alert: BackendTargetDown
        expr: up{job="backend-api"} == 0
        for: 2m
        labels:
          severity: critical
          team: backend
        annotations:
          summary: "Backend API is unreachable by Prometheus"
          description: "Inspect target health, application logs, networking, and recent
deployments."

  - name: backend-performance
    rules:
      - alert: BackendHigh5xxRate
        expr: |
          (
            sum(rate(http_server_requests_seconds_count{
              job="backend-api",
              status=~"5.."
            }[5m]))
            /
            clamp_min(
              sum(rate(http_server_requests_seconds_count{
                job="backend-api"
              }[5m])),
              0.001
            )
          ) > 0.05
        for: 5m
        labels:
          severity: warning
          team: backend
        annotations:
          summary: "Backend server-error ratio is elevated"
          description: "More than 5% of observed requests are returning 5xx responses."
```

Important: The metric name and labels depend on your Spring Boot and Micrometer configuration. Verify the actual series at `/actuator/prometheus` before using these rules. A rule that references nonexistent metrics will not deliver the intended alert.

The 5% threshold is illustrative, not a recommended universal production threshold. Real thresholds should reflect the service's SLO, traffic volume, and acceptable error rate.

The denominator uses `clamp_min` to avoid division by zero, but that does not make a low-traffic ratio statistically reliable. Production rules should also account for minimum traffic volume and the service's actual reliability objective.

Step 3 — Validate the rules

Use Prometheus's rule validation tooling where available, and then inspect the running instance.

Useful checks:

```
promtool check rules monitoring/alert-rules.yml
```

After reloading or restarting Prometheus, inspect:

- The **Rules** page to confirm that the rules loaded.

- The **Alerts** page to see inactive, pending, and firing states.
- The **Targets** page to confirm that the backend is being scraped.
- The query interface to verify each expression independently.

A syntactically valid rule can still be logically incorrect. Always test its PromQL expression against real series.

7. Configure Alertmanager

Create `alertmanager.yml`:

```
route:
  receiver: default-receiver
  group_by: ["alertname", "team", "severity"]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h

receivers:
  - name: default-receiver
```

This defines routing and grouping, but it does not deliver a notification to an external service because no delivery integration has been configured.

For real notifications, add a supported receiver such as an email integration, webhook, or incident-management service. Store credentials securely and avoid committing them to Git.

A minimal Compose extension could include:

```
services:
  alertmanager:
    image: prom/alertmanager:latest
    command:
      - "--config.file=/etc/alertmanager/alertmanager.yml"
    volumes:
      - ./monitoring/alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro
    ports:
      - "9093:9093"
```

This assumes the configuration file exists and that the service joins the relevant Compose network. Production deployments should pin image versions rather than relying on `latest`, and should use deliberate network and port exposure policies.

If Prometheus is running in a container, configure its alerting section to reach Alertmanager by the Compose service name:

```
alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]
```

The `alerting` section belongs in `prometheus.yml`, at the top level. Ensure the Alertmanager service is reachable from Prometheus.

Alert lifecycle walkthrough

1. A backend metric changes.
2. Prometheus scrapes the metric.
3. The alert expression evaluates to true.
4. The alert waits through its configured `for` period, if any.
5. Prometheus sends the firing alert to Alertmanager.
6. Alertmanager groups and routes it.
7. A configured receiver delivers the notification.

Test this complete chain. Seeing an alert marked as firing in Prometheus does not prove the notification reached the on-call engineer.

8. Recording Rules: Precompute Expensive Queries

A recording rule periodically evaluates a PromQL expression and stores the result as a new time series.

Why is this useful?

A dashboard may repeatedly calculate the same expensive aggregation across many application instances. Instead of running that full expression for every dashboard refresh, Prometheus can calculate it on a schedule.

Example:

```
groups:
  - name: backend-recording-rules
    interval: 30s
    rules:
      - record: job:http_requests:rate5m
        expr: |
          sum by (job) (
            rate(http_server_requests_seconds_count[5m])
          )
```

You can then query the recorded result:

```
job:http_requests:rate5m
```

Recording rules are useful for:

- Frequently used dashboard queries.
- Repeated aggregations across many instances.
- Standardized service-level metrics.
- Reducing the cost of complex queries.

However, recording rules consume storage and evaluation resources. They also introduce a refresh interval, so the recorded result is not necessarily identical to evaluating the original expression at the current instant.

Naming conventions

A common style is:

```
level:metric:operations
```

For example:

```
job:http_requests:rate5m
```

The name should communicate the aggregation level, metric, and operation. Consistency is more important than mechanically following a convention without understanding it.

When not to use recording rules

Avoid creating recording rules for every query. Prefer them when a query is frequently reused or expensive enough to justify precomputation.

Before adding one:

1. Measure the query's cost.
2. Check whether the result is reused.
3. Choose a suitable evaluation interval.
4. Confirm the new series has controlled cardinality.
5. Verify that dashboards and alerts use the recorded series correctly.

9. Design Useful SLIs and SLOs

Consider a Spring Boot Orders API.

Possible SLIs include:

- Availability: proportion of eligible requests that succeed.
- Latency: proportion of eligible requests completed within a defined threshold.
- Correctness: proportion of operations that produce valid outcomes, if correctness can be measured reliably.

For a request-based availability SLI:

```
sum(rate(http_server_requests_seconds_count{
  job="backend-api",
  status!~"5.."
}[5m]))
/
sum(rate(http_server_requests_seconds_count{
  job="backend-api"
}[5m]))
```

This is only a basic example. The definition of success must match the service contract: not every non-5xx response necessarily means a request was successful from the user's perspective. Exclusions such as health checks and expected client errors should be defined deliberately.

For a latency objective using a histogram, suppose the target is that 95% of requests complete within 300 ms:

```
sum(rate(http_server_requests_seconds_bucket{
  job="backend-api",
  le="0.3"
}[5m]))
/
sum(rate(http_server_requests_seconds_count{
  job="backend-api"
}[5m]))
```

This computes the observed fraction of requests in the histogram bucket at or below 0.3 seconds, assuming the histogram's buckets and count series are configured consistently.

It is often more useful to measure the fraction of requests that meet the latency objective than to alert on a percentile alone.

Why? A percentile can obscure the number of users affected, particularly at low traffic volumes. A good SLI definition makes the user impact explicit.

10. Error Budgets and Burn Rates

An error budget describes how much unreliability is allowed by the SLO. A burn rate measures how quickly that budget is being consumed relative to the target.

For an availability SLO of 99.9%, the allowed error ratio is:

Formula: 1 - 0.999 = 0.001

If the observed error ratio is 0.01, then:

Formula: Burn rate = (0.01 / 0.001) = 10

The service is consuming its error budget at ten times the sustainable rate during that measurement window.

A high burn rate can justify paging even when the raw error percentage looks modest. Conversely, a small absolute number of errors at low traffic may not justify an immediate page.

Production alerting commonly evaluates different windows:

- A short window catches sudden incidents.
- A longer window reduces sensitivity to brief spikes.
- Combining windows can distinguish sustained degradation from transient noise.

Exact windows and thresholds depend on the SLO, traffic, and response expectations. Do not copy generic burn-rate thresholds without validating them against your service.

11. Debugging Prometheus Rules and Alerts

Use this checklist when an expected alert does not fire.

A. Confirm the target is being scraped

```
up{job="backend-api"}
```

If the series is absent, investigate target discovery and scrape configuration. If it exists with value `0`, inspect connectivity, endpoint paths, authentication, and application availability.

B. Confirm the metric exists

```
{job="backend-api"}
```

Use the query browser and metrics endpoint to discover actual names and labels. Avoid assuming a metric exists because an example online uses it.

C. Run the alert expression manually

Evaluate the exact `expr` in the Prometheus query interface. Check whether it returns the series and values you expect.

D. Inspect rule state

Confirm the rule loaded and determine whether it is inactive, pending, or firing. A `for` duration may explain why a true condition has not yet fired.

E. Inspect Alertmanager

If Prometheus shows a firing alert but no notification arrives, verify:

- Prometheus can reach Alertmanager.
- Alertmanager has loaded its configuration.
- The alert matches a route.
- A receiver is configured correctly.
- Delivery credentials and external service settings are valid.
- The alert is not silenced or inhibited.

F. Check time and labels

Unexpected timestamps, inconsistent labels, aggregation mistakes, or an overly restrictive matcher can cause alerts to disappear from a query or route.

Layer 3 — Production Engineering & Interview Mastery

12. Production Alerting Principles

Principle 1: Every page needs an action

A page should make it reasonably clear what the responder should investigate or do.

Weak annotation:

```
summary: "High CPU"
```

Better annotation:

```
summary: "Orders API CPU saturation is affecting request latency"
description: "Check request latency, recent deployments, CPU throttling, and thread-pool
 saturation for the Orders API."
```

The second annotation offers context and initial diagnostic direction. It does not claim that CPU is necessarily the root cause.

Principle 2: Separate pages from tickets

A page is for urgent, actionable conditions requiring timely human response. Lower-priority conditions can be routed to a ticket, dashboard, or periodic review.

Paging every time a noncritical threshold is crossed creates alert fatigue. Excessive alert volume makes genuinely urgent incidents easier to miss.

Principle 3: Avoid one-size-fits-all thresholds

A CPU threshold suitable for one service may be irrelevant to another. Consider:

- Baseline and expected workload.
- User-visible latency and error rates.
- Traffic volume.
- Recovery time.
- SLOs and business impact.
- The cost of a missed alert versus a false alarm.

Principle 4: Monitor the monitoring system

Prometheus can fail, become overloaded, lose connectivity, or stop evaluating rules correctly.

Monitor the monitoring pipeline itself:

- Prometheus target health.
- Rule evaluation failures and duration.
- Storage capacity and retention.
- Prometheus process health.
- Alertmanager health and notification failures.
- Gaps in expected metric ingestion.

An alerting system that silently stops working can create a false sense of safety.

13. Cardinality and Scaling Risks

Prometheus creates a distinct time series for each unique combination of metric name and label values.

Consider:

```
http_requests_total{method="GET",status="200",instance="api-1"}
```

Adding a label such as `user_id` can produce a large number of unique combinations. The problem becomes worse when multiple high-cardinality labels are combined.

Avoid labels containing unbounded or highly variable values, such as:

- User IDs.
- Request IDs.
- Raw URLs containing unique identifiers.
- Email addresses.
- Arbitrary error messages.

Prefer bounded labels such as HTTP method, normalized route, status class, and service name.

Use logs or traces for high-cardinality diagnostic details. Metrics should efficiently answer aggregate operational questions.

Production safeguards

- Establish a metric and label naming standard.
- Review instrumentation before release.
- Watch active-series growth.
- Set appropriate retention and storage capacity.
- Avoid scraping unnecessarily expensive endpoints.
- Use recording rules for justified recurring aggregations.
- Keep labels stable across deployments where possible.
- Evaluate the cost of every new metric family.

Do not treat dropping metrics indiscriminately as the first response to storage pressure. Identify the cardinality source and preserve the measurements required for service reliability.

14. Security and Reliability

Prometheus metrics can expose internal architecture, endpoint names, workload behavior, and operational details. Monitoring endpoints should not automatically be public.

Production safeguards include:

- Restricting access to Prometheus and Alertmanager interfaces.
- Exposing only necessary application endpoints.
- Using network policies, firewalls, and authentication where appropriate.
- Keeping receiver credentials outside source control.
- Using TLS where required by the deployment environment.
- Controlling who can create silences or change routing.
- Testing configuration changes before applying them.
- Keeping versions and upgrades under change control.
- Planning storage, retention, and recovery procedures.

Prometheus's pull architecture is operationally convenient, but it is not a complete security boundary. Network reachability and endpoint permissions still matter.

Configuration and deployment discipline

For monitoring configuration changes:

1. Review the diff.
2. Validate syntax and rules.
3. Test important expressions.
4. Confirm target labels and routing behavior.
5. Deploy through a controlled workflow.
6. Verify the new configuration is loaded.
7. Confirm expected alerts still fire and clear correctly.

A monitoring change can be a production change. A typo in an alert expression may silently remove coverage for an important failure.

15. Production Incident Simulations

Incident A — API is down, but there is no page

Symptoms: The API is unavailable, but the on-call engineer receives no notification.

Investigation:

1. Check whether Prometheus itself is healthy.
2. Inspect `up{job="backend-api"}`.
3. Confirm the target is present in the scrape configuration.
4. Check whether the alert rule loaded.
5. Evaluate the rule expression manually.
6. Inspect pending and firing state.
7. Check Prometheus-to-Alertmanager connectivity.
8. Inspect routing, silences, and receiver configuration.

Likely causes: Missing target discovery, incorrect job labels, a bad expression, a failed rule load, or a notification-routing problem.

Engineering lesson: A healthy dashboard is not proof that the alerting path is healthy.

Incident B — Every deployment causes hundreds of alerts

Symptoms: A routine rollout produces a burst of notifications.

Investigation:

1. Group alerts by alert name, service, and team.
2. Check whether many instance-level alerts share one underlying cause.
3. Inspect deployment and scrape timing.
4. Determine whether the alert threshold is too sensitive to short-lived transitions.
5. Review whether a `for` duration is appropriate.
6. Configure grouping and inhibition carefully.
7. Ensure a genuine prolonged outage still generates an actionable notification.

Likely fixes: Better grouping, more meaningful alert scope, an appropriate pending duration, and improved rollout-aware monitoring.

Caution: Do not suppress all alerts during deployment. A rollout can create a real outage that needs intervention.

Incident C — Prometheus storage grows unexpectedly

Symptoms: Disk usage rises rapidly and Prometheus begins experiencing storage pressure.

Investigation:

1. Inspect disk capacity and Prometheus logs.
2. Review active-series growth and ingestion volume.
3. Identify recently introduced metrics and labels.
4. Check for unbounded values in labels.
5. Review scrape frequency and target count.
6. Examine retention and storage configuration.
7. Estimate the effect of corrective changes before deployment.

Likely fix: Remove or redesign unnecessary high-cardinality instrumentation, then tune retention and storage capacity according to the monitoring requirements.

Engineering lesson: Metrics design is a resource-management decision, not merely an instrumentation decision.

Incident D — The dashboard is green, but users report slow requests

Symptoms: A service dashboard appears healthy, yet customers report elevated latency.

Investigation:

1. Confirm the dashboard covers the affected route and user traffic.
2. Inspect request-duration histograms and traffic distribution.
3. Compare recent deployments and dependency latency.
4. Check whether the dashboard uses averages that hide tail latency.
5. Verify that the instrumentation is present and current.
6. Examine database, Redis, and downstream-service metrics.
7. Compare observed behavior with the defined latency SLO.

Likely causes: Incomplete instrumentation, overly broad aggregation, a dashboard that hides tail latency, or a slow dependency.

Engineering lesson: A dashboard is a view of selected measurements, not proof of service health.

16. Prometheus Interview Questions and Answers

Fundamentals

1. What is Prometheus?

Prometheus is a monitoring and time-series database system that collects metrics, stores time-stamped samples, supports PromQL queries, and evaluates alerting and recording rules.

2. How does Prometheus collect metrics?

Typically, it scrapes HTTP endpoints exposed by applications or exporters at configured intervals.

3. What is a time series?

A named metric identified by its label set, with timestamped sample values.

4. What is cardinality?

The number of distinct time series. Cardinality can grow rapidly when metrics have many label combinations.

5. What is PromQL?

Prometheus's query language for selecting, aggregating, and calculating over time-series data.

Alerting

6. What is the difference between Prometheus and Alertmanager?

Prometheus collects and queries metrics and evaluates rules. Alertmanager groups, deduplicates, inhibits, silences, and routes alerts to notification receivers.

7. What does `for: 5m` mean in an alert rule?

The condition must remain true for five minutes before the alert becomes firing.

8. What is the difference between an alerting rule and a recording rule?

An alerting rule creates an alert when an expression meets a condition. A recording rule evaluates an expression periodically and stores the result as a new time series.

9. What is alert fatigue?

A situation where excessive or noisy notifications reduce the likelihood that responders notice and act on important alerts.

10. How do you debug an alert that does not fire?

Check target health, metric existence, the expression, rule loading, pending duration, labels, and Alertmanager routing.

Reliability and SLOs

11. What is an SLI?

A quantitative indicator of service behavior, such as successful-request ratio or the proportion of requests meeting a latency threshold.

12. What is an SLO?

A target for an SLI over a defined measurement window.

13. What is an error budget?

The permitted unreliability implied by an SLO over its measurement window.

14. What is a burn rate?

The rate at which a service consumes its error budget relative to the sustainable rate defined by the SLO.

15. Why can a 5xx-rate alert be misleading at low traffic?

A small number of failures can create a high ratio, while the ratio may also be based on too few observations to be stable. Traffic volume and user impact should inform the alert design.

Production Engineering

16. Why should user IDs generally not be Prometheus labels?

They can create an enormous number of unique time series, increasing memory, storage, and query costs.

17. What are recording rules used for?

They precompute and store frequently reused or expensive expressions, making dashboards and alerts more efficient.

18. Why should you monitor Prometheus itself?

Because collection, rule evaluation, storage, or notification delivery can fail, leaving the team without reliable visibility.

19. How would you investigate a sudden increase in Prometheus disk usage?

Inspect ingestion and active-series growth, review recent metric and label changes, identify high-cardinality series, and check retention and storage settings.

20. Why is a symptom-oriented alert often more useful than a CPU alert?

A symptom such as a rising request-error ratio directly indicates user-visible degradation. CPU utilization is a diagnostic signal whose significance depends on workload and service behavior.

17. Hands-On Capstone

Build a monitoring and alerting workflow for a Spring Boot Orders API.

Required tasks

- [ ] Expose application metrics through Micrometer and Spring Boot Actuator.
- [ ] Configure Prometheus scraping.
- [ ] Verify the target is healthy.
- [ ] Write an alert for an unavailable backend target.
- [ ] Write a service-error-ratio alert using the actual metrics available.
- [ ] Configure Alertmanager with at least one real or local test receiver.
- [ ] Confirm that a firing alert reaches the receiver.
- [ ] Create a recording rule for a frequently used aggregation.
- [ ] Define one availability SLI and one latency SLI.
- [ ] Specify an SLO and explain its error budget.
- [ ] Simulate a missing metric or unreachable target.
- [ ] Diagnose a notification that fails to arrive.
- [ ] Review metric labels for cardinality risks.
- [ ] Document the incident, evidence, root cause, and preventive action.

Completion standard

You should be able to explain the entire path from a backend metric to a human notification, distinguish a real service-level objective from a dashboard threshold, and troubleshoot a failure without relying on guesswork.

Final Mastery Checklist

Before marking Prometheus complete, verify that you can:

- [ ] Explain Prometheus's pull-based collection model.
- [ ] Distinguish counters, gauges, histograms, and summaries.
- [ ] Write and validate PromQL expressions.
- [ ] Explain alerting-rule states and the `for` duration.
- [ ] Describe the division of responsibility between Prometheus and Alertmanager.
- [ ] Configure grouping, routing, and notification delivery.
- [ ] Explain SLIs, SLOs, SLAs, and error budgets.
- [ ] Explain recording rules and when to use them.
- [ ] Diagnose missing targets, metrics, rules, and notifications.
- [ ] Recognize high-cardinality instrumentation.
- [ ] Design actionable alerts with clear responder guidance.
- [ ] Explain how to monitor the monitoring system.
- [ ] Complete the capstone and defend the design trade-offs.

Prometheus Handbook Completion Protocol

This is the second and final planned Prometheus content chunk. The Prometheus curriculum is now **complete in authored content**.

Repository and release work remain separate:

1. Save this chapter as `content/prometheus/02-prometheus-alerting-slos-recording-rules-production-reliability.md`.
2. Verify that Chunk 1 and Chunk 2 both exist in `content/prometheus/` and contain the complete source content.
3. Review the combined handbook for consistency, working examples, and accurate metric names.
4. Generate the **single complete Prometheus PDF**, not separate PDFs per chunk, at `docs/prometheus/BEH_PROMETHEUS_COMPLETE_HANDBOOK.pdf`.
5. Open and verify the PDF, including its examples, formatting, and completeness.
6. Update the README using verified progress only.
7. After the source files and final PDF are saved and verified, commit with the exact message:

```
Prometheus done
```

Do not mark the repository, PDF, or Git commit complete until those steps have actually been executed and verified.